Sentiment Analysis of Amazon Product Reviews

This notebook uses spaCy and spaCyTextBlob to analyse the sentiment of Amazon product reviews. Reviews are cleaned by converting the text to lowercase, removing stop words, punctuation and unnecessary whitespace.

The resulting sentiment polarity is classified as positive, negative or neutral.

Import Libraries

In [20]:
```python
import pandas as pd
import spacy

from spacytextblob.spacytextblob import SpacyTextBlob
```

Load the language model

In [21]:
```python
# Load the medium English spaCy model
nlp = spacy.load("en_core_web_md")

# Add the spaCyTextBlob sentiment component to the pipeline
if "spacytextblob" not in nlp.pipe_names:
    nlp.add_pipe("spacytextblob")

print("Pipeline components:", nlp.pipe_names)
```

```
Pipeline components: ['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer', 'ner', 'spacytextblob']
```

Load the customer reviews dataset

In [22]:
```python
# Load the Amazon product reviews dataset
dataframe = pd.read_csv("Datafiniti_Amazon_Consumer_Reviews_of_Amazon_Products_May19.csv")

print("Dataset shape:", dataframe.shape)
dataframe.head()
```

```
Dataset shape: (28332, 24)
```

Out[22]:

	id	dateAdded	dateUpdated	name	asins	brand	categories	primaryCategories	imageURLs	
0	AVpgNzjwLJeJML43Kpxn	2015-10-30T08:59:32Z	2019-04-25T09:08:16Z	AmazonBasics AAA Performance Alkaline Batterie...	B00QWO9P0O,B00LH3DMUO	Amazonbasics	AA,AAA,Health,Electronics,Health & Household,C...	Health & Beauty	https://images-na.ssl-images-amazon.com/images...	amazonb
1	AVpgNzjwLJeJML43Kpxn	2015-10-30T08:59:32Z	2019-04-25T09:08:16Z	AmazonBasics AAA Performance Alkaline Batterie...	B00QWO9P0O,B00LH3DMUO	Amazonbasics	AA,AAA,Health,Electronics,Health & Household,C...	Health & Beauty	https://images-na.ssl-images-amazon.com/images...	amazonb
2	AVpgNzjwLJeJML43Kpxn	2015-10-30T08:59:32Z	2019-04-25T09:08:16Z	AmazonBasics AAA Performance Alkaline Batterie...	B00QWO9P0O,B00LH3DMUO	Amazonbasics	AA,AAA,Health,Electronics,Health & Household,C...	Health & Beauty	https://images-na.ssl-images-amazon.com/images...	amazonb
3	AVpgNzjwLJeJML43Kpxn	2015-10-30T08:59:32Z	2019-04-25T09:08:16Z	AmazonBasics AAA Performance Alkaline Batterie...	B00QWO9P0O,B00LH3DMUO	Amazonbasics	AA,AAA,Health,Electronics,Health & Household,C...	Health & Beauty	https://images-na.ssl-images-amazon.com/images...	amazonb
4	AVpgNzjwLJeJML43Kpxn	2015-10-30T08:59:32Z	2019-04-25T09:08:16Z	AmazonBasics AAA Performance Alkaline Batterie...	B00QWO9P0O,B00LH3DMUO	Amazonbasics	AA,AAA,Health,Electronics,Health & Household,C...	Health & Beauty	https://images-na.ssl-images-amazon.com/images...	amazonb

5 rows × 24 columns

Inspect the dataset

In [23]:
```python
# Display the available columns
print("Dataset columns:")
print(dataframe.columns.tolist())

# Check missing values in the review column
print("\nMissing review texts:", dataframe["reviews.text"].isna().sum())
```

```
Dataset columns:
['id', 'dateAdded', 'dateUpdated', 'name', 'asins', 'brand', 'categories', 'primaryCategories', 'imageURLs', 'keys', 'manufacturer', 'manufacturerNumber', 'reviews.date', '
reviews.dateSeen', 'reviews.didPurchase', 'reviews.doRecommend', 'reviews.id', 'reviews.numHelpful', 'reviews.rating', 'reviews.sourceURLs', 'reviews.text', 'reviews.titl
e', 'reviews.username', 'sourceURLs']

Missing review texts: 0
```

Select and clean the relevant data

In [24]:
```python
# Remove rows where the review text is missing
clean_data = dataframe.dropna(
    subset=["reviews.text"]
).copy()

# Select the review text as the feature data
reviews_data = clean_data["reviews.text"].astype(str)

print("Original number of rows:", len(dataframe))
print("Rows after removing missing reviews:", len(clean_data))
```

```
Original number of rows: 28332
Rows after removing missing reviews: 28332
```

Datafiniti Amazon Consumer Reviews is the name of the dataset used that includes **28,332 product reviews** and has 24 features such as product names, brands, review ratings, titles and review texts. As a result, `reviews.text` was selected as the feature for sentiment analysis.

No missing values in the `reviews.text` feature were found; therefore, all 28,332 reviews were kept using `dropna()` function.

Prior to sentiment analysis, reviews were made lowercase and processed by spaCy. Stop words, punctuations and spaces were deleted, while words were lemmatized. For instance, the first review became:

`order 3 item bad quality miss backup spring pc aluminum battery work`

It represents a shorter string of the words that are deemed more important for the analysis.

Define the preprocessing function

In [26]:
```python
def preprocess_review(review_text):
    """
    Clean a product review using spaCy.

    The function:
    - converts the review to lowercase;
    - removes leading and trailing whitespace;
    - removes stop words, punctuation and spaces;
    - uses the lemma of each remaining word.
    """

    cleaned_text = str(review_text).lower().strip()
    document = nlp(cleaned_text)

    useful_tokens = []

    for token in document:
        if (
            not token.is_stop
            and not token.is_punct
            and not token.is_space
        ):
            useful_tokens.append(token.lemma_)

    return " ".join(useful_tokens)
```

Test the preprocessing

In [27]:
```python
sample_review = reviews_data.iloc[0]

print("Original review:")
print(sample_review)

print("\nProcessed review:")
print(preprocess_review(sample_review))
```

```
Original review:
I order 3 of them and one of the item is bad quality. Is missing backup spring so I have to put a pcs of aluminum to make the battery work.

Processed review:
order 3 item bad quality miss backup spring pc aluminum battery work
```

Define the sentiment-analysis function

In [28]:
```python
def analyse_sentiment(review_text):
    """
    Predict the sentiment of a product review.

    Returns:
        tuple: sentiment label, polarity and subjectivity.
    """

    processed_review = preprocess_review(review_text)

    # Process the cleaned text using spaCy and spaCyTextBlob
    document = nlp(processed_review)

    polarity = document._.blob.polarity
    subjectivity = document._.blob.subjectivity

    # Convert the polarity score into a readable category
    if polarity > 0.05:
        sentiment_label = "Positive"
    elif polarity < -0.05:
        sentiment_label = "Negative"
    else:
        sentiment_label = "Neutral"

    return sentiment_label, polarity, subjectivity
```

Test the function on own reviews

In [29]:
```python
sample_reviews = [
    "This product is excellent and works perfectly. I love it!",
    "The item stopped working after two days, seriously such a waste of money.",
    "The package arrived yesterday and contained only three batteries."
]

for review in sample_reviews:
    sentiment, polarity, subjectivity = analyse_sentiment(review)

    print("Review:", review)
    print("Sentiment:", sentiment)
    print(f"Polarity: {polarity:.4f}")
    print(f"Subjectivity: {subjectivity:.4f}")
    print("-" * 70)
```

```
Review: This product is excellent and works perfectly. I love it!
Sentiment: Positive
Polarity: 0.7500
Subjectivity: 0.8000
----------------------------------------------------------------------
Review: The item stopped working after two days, seriously such a waste of money.
Sentiment: Negative
Polarity: -0.2000
Subjectivity: 0.0000
----------------------------------------------------------------------
Review: The package arrived yesterday and contained only three batteries.
Sentiment: Neutral
Polarity: 0.0000
Subjectivity: 0.0000
----------------------------------------------------------------------
```

Test reviews from CSV

In [30]:
```python
# Select three reviews from the dataset
dataset_samples = reviews_data.iloc[:3]

for review_number, review in enumerate(dataset_samples, start=1):
    sentiment, polarity, subjectivity = analyse_sentiment(review)

    print(f"Review {review_number}:")
    print(review)
    print("Predicted sentiment:", sentiment)
    print(f"Polarity: {polarity:.4f}")
    print(f"Subjectivity: {subjectivity:.4f}")
    print("-" * 70)
```

```
Review 1:
I order 3 of them and one of the item is bad quality. Is missing backup spring so I have to put a pcs of aluminum to make the battery work.
Predicted sentiment: Negative
Polarity: -0.7000
Subjectivity: 0.6667
----------------------------------------------------------------------
Review 2:
Bulk is always the less expensive way to go for products like these
Predicted sentiment: Negative
Polarity: -0.5000
Subjectivity: 0.7000
----------------------------------------------------------------------
Review 3:
Well they are not Duracell but for the price i am happy.
Predicted sentiment: Positive
Polarity: 0.8000
Subjectivity: 1.0000
----------------------------------------------------------------------
```

As we can see, the clearly positive review (own review) equals **0.7500** and the sentiment of the dissatisfied one equals **-0.2000**. The neutral review about the package and batteries got a polarity of **0.0000**.

The results produced by the model are also quite appropriate for the reviews chosen from the dataset. The first review about bad quality and the absence of something got a polarity of **-0.7000** and was classified as negative. The third review said that the customer liked the product and was classified as positive.

The problem is in the second review where bulk buying was cheaper but the review was classified as negative. Thus, individual words such as “less” can affect the polarity regardless of the whole meaning.

Analyse a sample of the dataset

Processing all reviewss from the dataset through spaCy will most likely take some time, so we'll proceed with a sample of 1000 reviews

In [31]:
```python
# Select a reproducible sample for analysis
analysis_sample = clean_data.sample(n=min(1000, len(clean_data)), random_state=42).copy()

# Apply sentiment analysis to each sampled review
sentiment_results = analysis_sample["reviews.text"].apply(analyse_sentiment)

analysis_sample[["predicted_sentiment", "polarity", "subjectivity"]] = pd.DataFrame(sentiment_results.tolist(), index=analysis_sample.index)

analysis_sample[
    [
        "reviews.text",
        "reviews.rating",
        "predicted_sentiment",
        "polarity",
        "subjectivity"
    ]
].head()
```

Out[31]:

	reviews.text	reviews.rating	predicted_sentiment	polarity	subjectivity
19947	Awesome tablet. I was amazed how fast it is. A...	5	Positive	0.525000	0.700000
7445	They don't last. USed in electronics (like com...	1	Neutral	0.025000	0.300000
3525	Thx.	5	Neutral	0.000000	0.000000
24122	kids love it EZ to use great Quality bought th...	5	Positive	0.538889	0.736111
25572	The kids feature is great. My 18 month old tak...	4	Positive	0.443750	0.512500

Display the sentiment totals

In [32]:
```python
sentiment_counts = (
    analysis_sample["predicted_sentiment"]
    .value_counts()
    .rename_axis("Sentiment")
    .reset_index(name="Number of reviews")
)

print("Predicted sentiment totals:")
print(sentiment_counts.to_string(index=False))
```

```
Predicted sentiment totals:
Sentiment  Number of reviews
 Positive                814
  Neutral                138
 Negative                 48
```

Compare predictions with review ratings

In [33]:
```python
def rating_to_sentiment(rating):
    """
    Convert an Amazon star rating to a sentiment category.
    """

    if rating >= 4:
        return "Positive"
    if rating <= 2:
        return "Negative"
    return "Neutral"


analysis_sample["rating_sentiment"] = (
    analysis_sample["reviews.rating"]
    .apply(rating_to_sentiment)
)

comparison_table = pd.crosstab(
    analysis_sample["rating_sentiment"],
    analysis_sample["predicted_sentiment"],
    rownames=["Sentiment based on rating"],
    colnames=["Predicted text sentiment"]
)

comparison_table
```

Out[33]:

Predicted text sentiment	**Negative**	**Neutral**	**Positive**
Sentiment based on rating			
Negative	9	15	26
Neutral	8	9	33
Positive	31	114	755

Calculate approximate agreement

In [34]:
```python
agreement = (
    analysis_sample["rating_sentiment"]
    == analysis_sample["predicted_sentiment"]
).mean()

print(
    f"Agreement between ratings and predicted sentiment: "
    f"{agreement:.2%}"
)
```

```
Agreement between ratings and predicted sentiment: 77.30%
```

Of the 1,000 reviews that were analyzed, 814 were identified as positive reviews, 138 as neutral and 48 as negative. The accuracy of the classification of the model was 77.3%.

The classifier performed quite well in analyzing positive reviews, while negative and neutral reviews were harder to identify correctly. In some cases, reviews that seemed negative at first glance were classified as neutral or positive. The reason behind this could be the shortness of some reviews, different opinion expression within one review, and absence of negations like “not” and “don’t”.

The ratings assigned to the products were used only as rough guidelines because the rating of a review and its sentiment are not always the same thing.

Display mismatched examples

In [35]:
```python
mismatched_reviews = analysis_sample[
    analysis_sample["rating_sentiment"]
    != analysis_sample["predicted_sentiment"]
][
    [
        "reviews.text",
        "reviews.rating",
        "rating_sentiment",
        "predicted_sentiment",
        "polarity"
    ]
]

print("Examples where rating and text sentiment differ:")

for _, row in mismatched_reviews.head(5).iterrows():
    print("\nReview:", row["reviews.text"])
    print("Rating:", row["reviews.rating"])
    print("Rating category:", row["rating_sentiment"])
    print("Predicted sentiment:", row["predicted_sentiment"])
    print(f"Polarity: {row['polarity']:.4f}")
    print("-" * 70)
```

```
Examples where rating and text sentiment differ:

Review: They don't last. USed in electronics (like computer mice, computer keyboards) Energizer or Duracell last easily 3x longer. Not worth the savings. If its going in so
me kids game or infrequent use (where the battery would be dead later anyways) ok.
Rating: 1
Rating category: Negative
Predicted sentiment: Neutral
Polarity: 0.0250
----------------------------------------------------------------------

Review: Thx.
Rating: 5
Rating category: Positive
Predicted sentiment: Neutral
Polarity: 0.0000
----------------------------------------------------------------------

Review: The 48 packs are the lowest price per AA battery you will find anywhere. I have a device I use for work that needs 8 new AA batteries every working day. These batte
ries get me through a 12 hour day the same as the expensive batteries. I could switch to rechargeable batteries but the simplicity of just putting in new batteries and not
fooling with recharging every night is worth it to me.
Rating: 5
Rating category: Positive
Predicted sentiment: Neutral
Polarity: 0.0145
----------------------------------------------------------------------

Review: I have a full load of batteries
Rating: 5
Rating category: Positive
Predicted sentiment: Neutral
Polarity: 0.0000
----------------------------------------------------------------------

Review: For a five year old this worked fine but was a bit slow loading games. But 5 year olds these days want it immediately. Like the kid friendly free books and games. I
t worked well with downloaded Disney movies and Netflix. May have to get one for 2 year old suster they both live it
Rating: 3
Rating category: Neutral
Predicted sentiment: Positive
Polarity: 0.0528
----------------------------------------------------------------------
```

Compare reviews using similarity

In [36]:
```python
def compare_reviews(first_review, second_review):
    """
    Calculate the semantic similarity between two reviews.
    """

    first_document = nlp(str(first_review))
    second_document = nlp(str(second_review))

    return first_document.similarity(second_document)
```

In [37]:
```python
first_review = reviews_data.iloc[0]
second_review = reviews_data.iloc[1]

similarity_score = compare_reviews(
    first_review,
    second_review
)

print("First review:")
print(first_review)

print("\nSecond review:")
print(second_review)

print(f"\nSimilarity score: {similarity_score:.4f}")
```

```
First review:
I order 3 of them and one of the item is bad quality. Is missing backup spring so I have to put a pcs of aluminum to make the battery work.

Second review:
Bulk is always the less expensive way to go for products like these

Similarity score: 0.9519
```

The two chosen reviews were given a semantic similarity score of **0.9519**. As the value is very close to 1, according to spaCy's criteria, the reviews are similar in terms of language or meaning.

The two reviews mention batteries, quality, and purchasing value; this probably contributes to their high degree of similarity. The two reviews have a different sentiment: while the first review criticizes the poor quality of the products, the second one stresses their cheaper price when buying in bulk.

This shows that semantic similarity is not the same as sentiment similarity, as two different opinions may be expressed regarding the same product or topic.

Here, the medium spaCy model is appropriate since it has word vectors that allow for semantic similarity comparison.

Final Conclusion

For the sentiment analysis in Amazon product reviews, the project used spaCy and spaCyTextBlob. There were 28,332 reviews in the dataset without any null values in the `reviews.text` field.

The sentiment function was able to accurately identify the positive and negative sentiments in a lot of the reviews. On a sampling of 1,000 reviews, it classified 814 as positive, 138 as neutral and 48 as negative. Comparing its sentiment predictions with the categories of sentiment based on product ratings, there was an approximate accuracy of **77.3%**.

The advantages of the model include simplicity of implementation, readability of polarity score and non-dependence on a training dataset. However, negation, sarcasm, short reviews, conflicting reviews and context-dependent phrases could prove difficult for the model. The removal of stop words might mean the removal of words needed for negation.

Similarity Analysis provided a value of **0.9519** for two reviews regarding batteries, which shows that spaCy managed to identify the fact that the reviews were similar in topic. The reviews gave dissimilar views, which shows that semantic similarity considers the topic and meaning of the reviews and not the sentiment of the reviews.

All in all, the model is able to provide helpful information on the sentiment of customers but one cannot take the predictions at face value.

Report done by: Ally Remas